Feuille d'Acquisition des Données (DAS) - pili-pos

Conformément au §II.C du Manuel de certification SCE 2.0, ce document décrit **où** les données fiscales et applicatives sont stockées, **comment** y accéder, **combien de temps** elles sont conservées et **comment** elles peuvent être exportées pour un contrôle SPF Finances.

1. Sources de données

1.1 Base de données fiscale principale

Élément	Valeur
Moteur	SQLite 3.x (bundled via `rusqlite`)
Mode	WAL (Write-Ahead Logging)
Chemin (production kiosque)	`/var/lib/pili-pos/pos.db`
Chemin (développement macOS)	`~/Library/Application Support/pili-pos/pos.db`
Chemin (développement Linux)	`~/.local/share/pili-pos/pos.db`
Encodage	UTF-8
Fichiers annexes WAL	`pos.db-wal`, `pos.db-shm` (mêmes répertoire et permissions)
Propriétaire	`pos:pos` (UID/GID dédiés sur le kiosque)
Permissions	`0640` (lecture restreinte au service)

Schéma complet : voir `SCHEMA_BD_FR.md`.

1.2 Sauvegardes automatiques de la base

Élément	Valeur
Déclencheur	Après chaque rapport Z, juste après son archivage (clôture journalière fiscale)
Mécanisme	API SQLite Online Backup (cohérent en WAL)
Répertoire	`<data_dir>/pili-pos/backups/` ou chemin configuré dans `PosConfig.backup_dir`
Format de nom	`pos-YYYYMMDD-HHMMSS.db`
Format	Fichier SQLite identique au fichier vivant
Permissions	`0640` (lecture restreinte)
Rétention	Indéfinie (au moins 7 ans, durée légale belge)

Implémentation : `src-tauri/src/commands/report.rs::backup_database_impl`.

1.3 Journaux applicatifs (logs structurés)

Élément	Valeur
Bibliothèque	`tracing` + `tracing-appender`
Format	Texte (une ligne par message, sans couleurs ; `tracing_subscriber::fmt`, `src-tauri/src/main.rs`)
Chemin (production)	`/var/lib/pili-pos/logs/pos.log.YYYY-MM-DD`
Rotation	Quotidienne, sans purge automatique
Niveaux	TRACE, DEBUG, INFO (défaut), WARN, ERROR

Élément	Valeur
Configuration runtime	Variable d'environnement `RUST_LOG` uniquement (fixée à `info` par `deploy/pili-pos.service`)

Les logs ne sont **pas une source d'information fiscale** - ils ne contiennent pas d'événements N/P/T/C/I/F/S/R, uniquement des événements applicatifs (démarrage, erreurs, alertes imprimante). Aucun INSZ n'y est écrit (§6). Les données fiscales authoritatives sont dans `pos.db::fiscal_events`.

1.4 Réponses brutes du FDM (audit)

Stockées **intégralement** dans la colonne `fiscal_events.fdm_response_json` (type TEXT, JSON sérialisé). Chaque ligne contient la réponse complète du FDM (incluant la signature ECDSA brute, l'URL de vérification, le QR data, les éventuelles informations/avertissements).

Ceci permet à un contrôleur de comparer chaque événement passé, signature ECDSA comprise, à l'enregistrement que le FDM a transmis au SPF Finances. Avant d'archiver une réponse, le POS a contrôlé qu'elle répond bien à sa requête (`crates/pos-fiscal/src/response_check.rs`).

1.5 Ticket numérique par e-mail (§III.H)

Deux tables de `pos.db`, lisibles comme le reste de la base (§2 ci-dessous) :

Table	Contenu	Ce qu'un contrôleur peut y lire
`ticket_deliveries`	Une ligne par ticket dont le support demandé incluait l'e-mail (`DIGITAL`/`PAPER_DIGITAL`) : support, texte du ticket envoyé, URL de vérification, statut d'envoi, nombre de tentatives, dernière erreur.	L'adresse en clair (`recipient`) n'y figure que pendant les envois/réessais en cours, et au plus 72 h (purge RGPD, cf. §6 ci-dessous). L'indice masqué (`recipient_hint`, ex. `j***@e***.be`) reste consultable indéfiniment.
`original_prints`	Une ligne par événement fiscal dont l'original a été imprimé, avec l'horodatage de l'impression.	Sert à vérifier qu'aucun original n'a été imprimé deux fois (POS spec v2.2 §8.1) - toute reproduction ultérieure porte un événement C séparé dans `fiscal_events`.

Voir `TICKET_NUMERIQUE_FR.md` pour le détail du cycle de vie de `ticket_deliveries` et `SCHEMA_BD_FR.md` §3.6-3.7 pour le schéma complet.

1.6 Données de configuration

Élément	Valeur
Fichier de configuration kiosque	`/etc/pili-pos/pos.json`
Contenu	identifiants POS (posId, terminalId, deviceId), URL FDM, adresse imprimante, infos restaurant, mode formation
Format	JSON
Modifiable par l'opérateur	Non (image scellée, accès root absent)
Source de vérité runtime	Dupliqué dans `AppState.config` au démarrage

2. Accessibilité

2.1 Sur la machine du commerçant

- Pas de console SSH active sur le kiosque (cf. `CHECKLIST_DEPLOIEMENT_KIOSK_FR.md`).
- L'opérateur ne dispose d'aucune commande shell : l'image démarre directement sur l'interface pili-pos en plein écran sous Cage Wayland.
- Le fichier de base de données est protégé en lecture (mode 0640, propriétaire `pos`).
- **Exception, image d'admission uniquement** : un compte `spf` (groupe `pos`, console `tty2`, mot de passe aléatoire affiché une seule fois au premier démarrage - `MANUEL_INSTALLATION_FR.md` §3.4) dispose du wrapper `pos-db-ro` pour interroger `pos.db` en lecture seule sans jamais pouvoir y écrire. Absent de l'image de production. Voir `SECURITE_BD_FR.md` §3.2 et §3.6.

2.2 Accès pour audit / contrôle SPF Finances

Quatre méthodes sont fournies (A, A bis, B, C) :

Méthode A - `pos-db-ro` sur le compte `spf` (recommandée, image d'admission) Sans démonter quoi que ce soit ni sortir le disque : sur la console `tty2` (`Ctrl+Alt+F2`), se connecter au compte `spf` (mot de passe noté par le distributeur au premier démarrage, `MANUEL_INSTALLATION_FR.md` §3.4) et lancer :

```
pos-db-ro                                    # shell SQL interactif
pos-db-ro "SELECT * FROM fiscal_events ORDER BY id DESC LIMIT 20;"
```

`pos-db-ro` (`/usr/local/bin/pos-db-ro`) ouvre `pos.db` en lecture seule (`sqlite3 -readonly`, ou `?immutable=1` si le service est arrêté) : le compte `spf` n'a physiquement pas la permission d'écrire dans le fichier (groupe `pos`, mode `0640` - `SECURITE_BD_FR.md` §3.1-3.2). Cette méthode n'existe que sur l'image d'admission ; sur un terminal en production, utiliser la méthode B ou contacter le distributeur.

Méthode A bis - Extraction physique du disque (accès de dernier recours) Depuis l'image mkosi, `/var` (donc `pos.db`) est chiffré **LUKS2** avec déverrouillage lié au TPM2 de ce terminal précis (`SECURITE_BD_FR.md` §3.6). Démonter le disque et le brancher ailleurs ne donne **pas** accès aux données : il faut soit le TPM2 d'origine (donc rebooter ce terminal précis), soit la **clé de récupération** notée par le distributeur au premier démarrage (`MANUEL_INSTALLATION_FR.md` §3.4) :

```
# Sur une machine de maintenance, disque du kiosque branché en externe
cryptsetup luksOpen /dev/sdX2 pili-data-recovery   # demande la clé de récupération
mount /dev/mapper/pili-data-recovery /mnt
sqlite3 -readonly /mnt/lib/pili-pos/pos.db
```

La clé de récupération est communiquée sur demande par le distributeur (`nuagemagique.dev`) ou, pour la machine de test SPF, disponible dans le dossier remis à l'admission.

Méthode B - Export via commande dédiée La commande Tauri `backup_database` (`src-tauri/src/commands/report.` produit à la demande un instantané cohérent (Online Backup API) dans le répertoire `backups/`. Un contrôleur peut demander à l'opérateur de déclencher un export juste avant inspection.

Méthode C - Requêtes SQL sur la copie Le POS ne fournit pas de commande d'export dédiée en CSV ou JSON. L'export est la base elle-même (méthodes A, A bis ou B), interrogée en SQL standard ; le contrôleur filtre par plage de dates et par type d'événement avec les requêtes du §2.3, sans rien modifier.

2.3 Format de sortie de l'export

Une fois la base copiée, les données s'interrogent via SQL standard :

```
-- Toutes les ventes (Événement N) d'une journée donnée
SELECT id, sequence_number, order_id, user_insz, created_at,
```

```sql
       fdm_id, event_label, short_signature
FROM fiscal_events
WHERE event_type = 'N'
  AND DATE(created_at) = '2026-10-03'
ORDER BY sequence_number;

-- Détail d'un événement spécifique (request + response complets)
SELECT event_data_json, fdm_response_json
FROM fiscal_events
WHERE id = 42;

-- Récap caisse d'une journée (totaux par mode de paiement)
SELECT method, SUM(amount_cents) / 100.0 AS total_eur, COUNT(*) AS nb
FROM payments
WHERE DATE(created_at) = '2026-10-03'
GROUP BY method;
```

3. Rétention

Donnée	Durée de rétention	Justification
`fiscal_events`	Indéfinie	Code Belge des Impôts - 7 ans minimum, jamais purgé
Sauvegardes quotidiennes (`pos-*.db`)	Indéfinie	Idem - chaque jour est un instantané indépendant
Logs applicatifs	Indéfinie (rotation quotidienne sans purge)	Permettre un audit de comportement applicatif a posteriori
`payments`, `orders`, `order_lines`	Indéfinie	Données de support nécessaires à la reconstruction d'un ticket
Translations, menu_items, modifier_groups, modifiers	Indéfinie	Données de référence. Le menu est mis à jour en place (y compris par la synchronisation du catalogue, qui désactive sans supprimer) ; l'historique fiscal n'en dépend pas : nom, prix et TVA sont figés dans `order_lines` et `order_entries`
`orders.raw_hubrise_json`, `hubrise_events` (commandes en ligne)	Indéfinie, sans purge automatique	Contiennent des données de clients (§6)

Aucune purge automatique n'est implémentée. Le passage à un modèle de purge légalement nécessitera une modification certifiée soumise au SPF Finances.

4. Protection contre la suppression / modification

- Cf. `DECLARATION_FONCTIONS_INTERDITES_FR.md` §1 et §2.
- La table `fiscal_events` ne fait l'objet d'aucun `DELETE`/`UPDATE` dans le code applicatif (vérifié par grep), et des triggers SQLite refusent toute suppression ou réécriture, même par un accès direct à la base (`fiscal_events_no_delete`, `fiscal_events_no_rewrite` ; idem `z_reports_*`, `order_entries_*`, `pending_signs_*`, `booking_periods_*` ; liste dans `SCHEMA_ER_FR.md` §4).
- Les sauvegardes journalières sont stockées en fichiers indépendants, jamais écrasés (le nom inclut le timestamp à la seconde).
- L'opérateur ne peut pas accéder au shell pour `rm` un fichier de données.

5. Intégrité

- WAL (Write-Ahead Logging) garantit l'atomicité crash-safe.
- Chaque opération fiscale est signée sous un verrou unique, puis archivée dans une transaction SQLite qui inclut compteur + archive + (le cas échéant) fermeture commande + clôture de la ligne `pending_signs`. Cf. `STRATEGIE_COMPTEURS_FR.md` §5.
- La signature ECDSA secp256r1 stockée dans `fiscal_events.fdm_response_json` est celle que le FDM a calculée et transmise au SPF Finances : une ligne modifiée rétroactivement ne correspondrait plus à l'événement signé détenu par le FDM et le SPF.
- Hash SHA-1 du binaire `pili-pos` consigné dans le formulaire de demande de certification, vérifiable sur le kiosque déployé via `sha1sum /opt/pili-pos/pili-pos`.

6. Confidentialité (GDPR)

- INSZ du personnel : numéro national, donnée personnelle identifiante.
- INSZ stocké dans les colonnes `users.insz`, `orders.user_insz`, `fiscal_events.user_insz`, `pending_signs.user_insz`, `staff_shifts.user_insz`, `webshop_orders.accepted_by`, `integration_switches.changed_by` et `integration_switch_log.changed_by`, ainsi que dans les JSON `fiscal_events.event_data_json` (payload envoyé au FDM, obligation fiscale), `pending_signs.request_body`, `pending_signs.archive_json` et `pending_signs.context_json`, et `z_reports.data_json` (rapport par utilisateur). Les rapports imprimés par utilisateur (X/Z) portent aussi l'INSZ.
- Aucun INSZ n'est écrit dans les journaux applicatifs : les messages qui concernaient un opérateur (vente soldée sans encaissement, session d'assistance à distance) n'enregistrent que son rôle, et aucun filtre de masquage n'est nécessaire.
- Pas de transmission de l'INSZ hors du périmètre POS+FDM (la synchronisation HubRise n'envoie ni INSZ ni nom d'employé). Voir `PROFIL_ENTREPRISE_FR.md` §GDPR pour le détail des bases légales (article 6.1.c GDPR - obligation légale).
- Adresse e-mail du ticket numérique (§1.5) : donnée personnelle fournie volontairement par le client, à la demande de celui-ci (base légale : demande du client, cf. `PROFIL_ENTREPRISE_FR.md` §6). Conservée en clair au maximum 72 h (`ticket_deliveries.recipient`), effacée immédiatement dès l'envoi réussi ou le repli papier ; seul un indice masqué survit indéfiniment. Jamais journalisée en clair, jamais affichée dans l'écran "Tickets numériques" des Paramètres (`list_ticket_deliveries` n'expose que l'indice masqué). Sous-traité à Scaleway (Transactional Email, UE, région `fr-par`) pour l'envoi uniquement - aucun usage marketing.
- Données de clients des commandes en ligne (si l'intégration HubRise est activée pour l'installation, désactivée par défaut) : `orders.raw_hubrise_json` et `hubrise_events.payload_json` conservent la commande telle que reçue, avec nom, téléphone et adresse ; le ticket client non fiscal les imprime ; les ventes de la caisse sont renvoyées à HubRise par `hubrise_outbox` (lignes, totaux, paiements, sans données de client). Aucune purge automatique. Le jeton d'accès HubRise (`hubrise_connection.access_token`) est stocké en clair dans `pos.db` : il est lisible par le compte `spf` de l'image d'admission (`pos-db-ro`) et présent dans les sauvegardes. Détail dans `SECURITE_BD_FR.md` §6.

7. Procédure d'export à fournir au contrôleur

Lors d'un contrôle, le distributeur ou son représentant fournit :

1. Un export complet de `pos.db` (par la Méthode A ou B ci-dessus).
2. Une copie du log `pos.log.*` couvrant la période contrôlée.
3. Le hash SHA-1 du binaire pili-pos en cours d'exécution sur le kiosque.

Le POS ne détient pas la clé publique du FDM : la vérification cryptographique des signatures se fait avec les données du FDM et du SPF Finances. Le contrôleur peut, à partir de l'export : - Comparer chaque événement (signature comprise) aux enregistrements du FDM et du SPF Finances. - Recalculer chaque shortSignature d'un événement N : Hex(SHA1(Base64Decode(digitalSignature))). - Reconstituer la séquence chronologique des événements. - Croiser avec les tickets imprimés (vérification QR code). - Repérer toute incohérence éventuelle entre l'archive locale et les enregistrements cloud SPF Finances.